Assignment 3

Learning goals:

Apply the concepts and methods presented in chapter "Parsing" of the lecture to some exercise problems.

Presentation:

Please prepare the solutions for informal presentation in class, and fill in the online check list in ZEUS.

Task 1 Pushdown automaton

Design a pushdown automaton for the following grammar.

```
G = x | a G b b
```

Explain how the input aaxbbbb is analyzed and show which states are stored on the stack in each step.

Task 2 Recursive descent parsing

Define Java parser methods for the following grammar of set expressions. ident is a terminal symbol. Note that implementation of an executable parser program is *not* required (i.e. no scanner or main method).

```
SetExpr = InterExpr { "∪"InterExpr } .
InterExpr  = DiffExpr { "∩" DiffExpr } .
DiffExpr = Field [ "\" Field ] .
Field = ident | "(" SetExpr ")"
```

Explain how your program parses the following inputs:

(a) (A ∪ B) ∩ C \ D (b) A \ B \ C ∪ D

Task 3 Recursive descent parsing

Write Java parser methods for the following grammar of object declarations with optional initialization. ident is a terminal symbol. You can assume that a parser method for Expr exists already. Note that implementation of an executable parser program is *not* required (i.e., no scanner or main method).

```
ObjDecl = ObjType ident [ "=" ObjInit ] ";" .
ObjType = ident .
ObjInit = "new" ObjType "(" [ObjArgs] ")" .
ObjArgs = Expr {"," Expr} .
```

Explain how your program parses the following input:

```
Vector3D v = new Vector3D(3,5,8);
```

Task 4 Parser implementation

Write an executable parser program for the following grammar of MicroJava (MJ) declarations:

```
Program = „program“ ident DeclList „{„ „}“.
DeclList = {ConstDecl | VarDecl | ClassDecl} .
ConstDecl = “final” Type ident “=” Literal “;” .
Literal = number | charConst .
Type = ident [ "[" "]" ].
VarDecl = Type ident { “,” ident} “;” .
ClassDecl = “class” ident “{“  { VarDecl  } “}” .
```

The parser program should take a MJ source file (consisting of declarations only) as input and check it for syntax errors with respect to the given grammar. The first detected error should be written to standard output, i.e. panic mode error handling is sufficient. For a syntactically valid source file the output should be “OK”.

Use the MJ.Scanner class of the MJ compiler provided in Moodle for scanning tokens. Note that the provided MJ.Parser class implements a complete compiler and should not be used for this task.

Task 5 Test parser implementation

Test your parser implementation of Task 4.

(a) Write 2 valid non-trivial MJ source files and verify that your parser program outputs “OK”.

(b) Write 3 syntactically invalid MJ source files and make sure that syntax errors are related to different productions of the given grammar. Then verify that your test program outputs the expected parse errors.

(c) Explain the syntax errors of invalid source files using the given grammar.